Running the Notification Stack with Docker

Commands for the backend services, Next.js frontend, and Angular frontend

Run all commands from the repository root in PowerShell.

1. First-time configuration

```
Copy-Item notification-api\.env.example notification-api\.env
notepad notification-api\.env
```

Fill in every value in `notification-api\.env`. Do not commit this file because it contains local credentials.

2. Start the backend stack

```
docker compose `
  --env-file notification-api\.env `
  --file notification-api\compose.week3.yml `
  up --build --detach --wait
```

This starts SQL Server, RabbitMQ, the Notification API, notification worker, DLQ recovery worker, and outbox worker.

3. Build and run Next.js

```
docker build `
  --tag nsa-next:local `
  --build-arg RELEASE_ID=local `
  .\next

docker run `
  --detach `
  --rm `
  --name nsa-next `
  --network nsa-week3-network `
  --publish 127.0.0.1:3000:3000 `
  --env NOTIFICATION_API_URL=http://api:8080 `
  --env RELEASE_COLOR=local `
  --env RELEASE_ID=local `
  nsa-next:local
```

4. Build and run Angular

```
docker build --tag nsa-angular:local .\angular

docker run `
  --detach `
```

```
  --rm `
  --name nsa-angular `
  --network nsa-week3-network `
  --publish 127.0.0.1:4200:8080 `
  nsa-angular:local
```

5. Application addresses

- Next.js: **http://localhost:3000**
- Angular: **http://localhost:4200**
- Notification API and Swagger: **http://localhost:8081/swagger**
- RabbitMQ Management: **http://localhost:15673**

6. Check container status

```
docker compose `
  --env-file notification-api\.env `
  --file notification-api\compose.week3.yml `
  ps

docker ps --filter "name=nsa-next" --filter "name=nsa-angular"
```

7. Stop everything

```
docker stop nsa-next nsa-angular

docker compose `
  --env-file notification-api\.env `
  --file notification-api\compose.week3.yml `
  down
```

Data safety: The shutdown command preserves SQL Server and RabbitMQ volumes. Add `--volumes` only when you intentionally want to erase persisted local data.